Time-Off Microservice - Test Strategy & Specifications

1. Test Architecture Overview

```
Test Pyramid:

          ┌──────────────────┐
          │   E2E Tests      │  15% - Full workflow scenarios
          │   (15-20 tests) │
          ├──────────────────┤
          │ Integration      │  45% - Service + DB + HCM Mock
          │ Tests            │  (30-40 tests)
          │ (30-40 tests)    │
          ├──────────────────┤
          │  Unit Tests      │  40% - Business logic, utilities
          │  (40-50 tests)   │  (40-50 tests)
          └──────────────────┘
```

Total Target: 85-110 test cases with >80% code coverage

---

2. Unit Tests (Service Layer)

2.1 BalanceService Tests (12 tests)

Test Suite: Balance Validation

1. **test_validateBalance_sufficient_returns_true**
   - Given: Employee has 10 days, requests 5 days
   - When: validateBalance called
   - Then: Returns true
2. **test_validateBalance_insufficient_returns_false**
   - Given: Employee has 3 days, requests 5 days
   - When: validateBalance called
   - Then: Returns false
3. **test_validateBalance_exact_amount_returns_true**
   - Given: Employee has exactly 5 days, requests 5 days

- When: validateBalance called
- Then: Returns true

4. **test_validateBalance_zero_balance_returns_false**
   - Given: Employee has 0 days
   - When: validateBalance called with 1 day request
   - Then: Returns false
5. **test_validateBalance_negative_balance_throws_error**
   - Given: Balance shows -2 days (corrupted data)
   - When: validateBalance called
   - Then: Throws DataIntegrityException

Test Suite: Balance Dimension Validation

6. **test_validateDimensions_valid_combo_returns_true**
   - Given: Valid employee-location-leaveType combination exists
   - When: validateDimensions called
   - Then: Returns true
7. **test_validateDimensions_invalid_employee_returns_false**
   - Given: Employee doesn't exist in system
   - When: validateDimensions called
   - Then: Returns false with error message
8. **test_validateDimensions_invalid_location_returns_false**
   - Given: Employee exists but location not assigned
   - When: validateDimensions called
   - Then: Returns false
9. **test_validateDimensions_invalid_leave_type_returns_false**
   - Given: Leave type doesn't exist in company policy
   - When: validateDimensions called
   - Then: Returns false

Test Suite: Cache Staleness Detection

10. **test_isCacheStale_fresh_cache_returns_false**
    - Given: Cache updated 5 minutes ago
    - When: isCacheStale called with TTL=30 minutes
    - Then: Returns false

11. **test_isCacheStale_expired_cache_returns_true**
    - Given: Cache updated 45 minutes ago
    - When: isCacheStale called with TTL=30 minutes
    - Then: Returns true
12. **test_isCacheStale_hcm_updated_after_cache_returns_true**
    - Given: HCM last updated at T1, cache at T0 (T0 < T1)
    - When: isCacheStale called
    - Then: Returns true (HCM data is newer)

2.2 TimeOffRequestService Tests (15 tests)

Test Suite: Request Creation

1. **test_createRequest_valid_request_returns_id**
    - Given: Valid request payload, sufficient balance
    - When: createRequest called
    - Then: Returns request ID and status PENDING
2. **test_createRequest_insufficient_balance_throws_error**
    - Given: Request for 10 days, balance only 5
    - When: createRequest called
    - Then: Throws InsufficientBalanceException
3. **test_createRequest_invalid_dimensions_throws_error**
    - Given: Invalid location for employee
    - When: createRequest called
    - Then: Throws DimensionValidationException
4. **test_createRequest_date_validation_invalid_dates_throws_error**
    - Given: Start date after end date
    - When: createRequest called
    - Then: Throws InvalidDateRangeException
5. **test_createRequest_past_dates_throws_error**
    - Given: Request for dates in the past
    - When: createRequest called
    - Then: Throws InvalidDateException

Test Suite: Request State Transitions

6. **test_approveRequest_pending_to_approved_succeeds**
   - Given: Request in PENDING state
   - When: approveRequest called
   - Then: State changes to APPROVED, approvedAt set
7. **test_approveRequest_non_pending_throws_error**
   - Given: Request in APPROVED state
   - When: approveRequest called again
   - Then: Throws InvalidStateTransitionException
8. **test_rejectRequest_pending_to_rejected_succeeds**
   - Given: Request in PENDING state
   - When: rejectRequest called with reason
   - Then: State changes to REJECTED, reason stored
9. **test_cancelRequest_approved_to_cancelled_succeeds**
   - Given: Request in APPROVED state
   - When: cancelRequest called
   - Then: State changes to CANCELLED
10. **test_cancelRequest_rejected_throws_error**
    - Given: Request in REJECTED state
    - When: cancelRequest called
    - Then: Throws InvalidStateTransitionException

Test Suite: Balance Snapshot

11. **test_createRequest_captures_balance_snapshot**
    - Given: Request created with 7 days available
    - When: Request stored
    - Then: Balance snapshot contains correct available balance
12. **test_approveRequest_uses_latest_balance_not_snapshot**
    - Given: Request created when balance was 7, now balance is 3
    - When: approveRequest called
    - Then: Uses current balance (3) not snapshot (7)

Test Suite: Conflict Detection

13. **test_approveRequest_detects_insufficient_balance_at_approval**
    - Given: Request created with sufficient balance, balance now insufficient

- When: approveRequest called
- Then: Throws BalanceConflictException with drift details

14. **test_approveRequest_concurrent_approval_conflict_detected**
    - Given: Two simultaneous approval requests for same balance
    - When: Both approveRequest called
    - Then: First succeeds, second gets version mismatch error
15. **test_approveRequest_marks_balance_as_pending_hcm_sync**
    - Given: Approval succeeds
    - When: No HCM confirmation yet
    - Then: Request status is APPROVED but hcm_sync_status is PENDING

2.3 SyncService Tests (10 tests)

Test Suite: Real-Time Sync

1. **test_syncBalanceFromHCM_success_updates_cache**
    - Given: HCM returns balance of 10 days
    - When: syncBalanceFromHCM called
    - Then: Local cache updated to 10 days, timestamp set
2. **test_syncBalanceFromHCM_timeout_throws_error**
    - Given: HCM API doesn't respond within 5 seconds
    - When: syncBalanceFromHCM called
    - Then: Throws HCMTimeoutException
3. **test_syncBalanceFromHCM_invalid_dimension_throws_error**
    - Given: HCM returns 404 for invalid employee-location combo
    - When: syncBalanceFromHCM called
    - Then: Throws DimensionNotFoundException

Test Suite: Batch Sync

4. **test_batchSyncBalances_multiple_employees_success**
    - Given: 3 employees with valid data in HCM
    - When: batchSyncBalances called
    - Then: All 3 employees' balances updated, return sync ID
5. **test_batchSyncBalances_partial_failure_continues**
    - Given: 3 employees, 1 fails (not found in HCM)

- When: batchSyncBalances called
- Then: 2 updated, 1 marked as error, sync completes

6. **test_batchSyncBalances_drift_detected_flags_as_stale**
   - Given: Local balance 10, HCM balance 15
   - When: batchSyncBalances called
   - Then: Balance updated to 15, is_stale marked false, history logged

Test Suite: Drift Detection

7. **test_detectDrift_hcm_increased_balance_detected**
   - Given: Local balance 10, HCM 15, HCM last updated after local
   - When: detectDrift called
   - Then: Drift detected, returns change details
8. **test_detectDrift_hcm_decreased_balance_alerts**
   - Given: Local balance 10, HCM 5, HCM last updated after local
   - When: detectDrift called
   - Then: Drift flagged as "critical", alert generated
9. **test_detectDrift_no_change_no_drift**
   - Given: Local balance 10, HCM 10, timestamps match
   - When: detectDrift called
   - Then: No drift detected
10. **test_detectDrift_multiple_balances_returns_all_changes**
    - Given: 5 employees, 2 have drift
    - When: detectDrift called
    - Then: Returns array of 2 drifted balances

2.4 RetryService Tests (8 tests)

1. **test_retryWithBackoff_succeeds_on_first_attempt**
   - Given: Function succeeds immediately
   - When: retryWithBackoff called with 3 max attempts
   - Then: Returns result on attempt 1
2. **test_retryWithBackoff_succeeds_on_second_attempt**
   - Given: Function fails once, succeeds on second call
   - When: retryWithBackoff called

- Then: Returns result on attempt 2, logs retry

3. **test_retryWithBackoff_fails_after_max_attempts**
   - Given: Function always fails
   - When: retryWithBackoff called with 3 max attempts
   - Then: Throws error after 3 attempts
4. **test_retryWithBackoff_exponential_delay_increases**
   - Given: Function fails 3 times
   - When: retryWithBackoff called
   - Then: Wait times are 100ms, 300ms, 900ms (exponential)
5. **test_retryWithBackoff_transient_error_retried**
   - Given: Function throws timeout error
   - When: retryWithBackoff with retryableErrors=[TimeoutError]
   - Then: Retries up to max attempts
6. **test_retryWithBackoff_permanent_error_fails_immediately**
   - Given: Function throws validation error
   - When: retryWithBackoff with nonRetryableErrors=[ValidationError]
   - Then: Throws immediately without retry
7. **test_retryWithBackoff_circuit_breaker_stops_retry**
   - Given: Circuit breaker open (too many failures)
   - When: retryWithBackoff called
   - Then: Throws CircuitBreakerOpenException
8. **test_retryWithBackoff_circuit_breaker_recovers**
   - Given: Circuit breaker open, but recovery timeout passed
   - When: retryWithBackoff called
   - Then: Tries again (circuit breaker half-open)

2.5 AuditService Tests (8 tests)

1. **test_logChange_creates_audit_entry**
   - Given: Balance change from 10 to 8
   - When: logChange called with BALANCE type
   - Then: Audit entry created with timestamps
2. **test_logChange_includes_old_and_new_values**

- Given: Balance change
- When: logChange called
- Then: Audit log contains both old_value and new_value as JSON

3. **test_logChange_identifies_change_source**
   - Given: Change source is "HCM_SYNC"
   - When: logChange called
   - Then: Audit log stores source correctly
4. **test_logChange_approves_chain_creates_multiple_entries**
   - Given: Request approval with balance deduction
   - When: Approval process completes
   - Then: Two audit logs created (request + balance)
5. **test_getAuditLog_retrieves_entity_history**
   - Given: Entity with 5 changes
   - When: getAuditLog called with entity_id
   - Then: Returns all 5 changes in chronological order
6. **test_getAuditLog_filters_by_date_range**
   - Given: 10 audit entries across 2 days
   - When: getAuditLog called with date range (1 day)
   - Then: Returns only entries in date range
7. **test_auditLog_immutable_cannot_delete**
   - Given: Audit entry exists
   - When: Delete attempted
   - Then: Operation fails, entry persists
8. **test_auditLog_includes_user_info**
   - Given: Change made by user "MGR-001"
   - When: logChange called with userId
   - Then: Audit log stores performed_by field

---

3. Integration Tests (Database + Service + API)

3.1 End-to-End Request Lifecycle (8 tests)

1. **test_create_and_approve_request_full_flow**
   - Setup: Employee with 10 days vacation balance

- When: Create request for 3 days → Approve request
- Then:
  - Request status = APPROVED
  - Balance reduced to 7 (local cache)
  - HCM sync status = SYNCED
  - Audit logs show both actions
- Verify: 2 audit entries, balance correctly updated

2. **test_create_request_with_insufficient_balance_rejected**
   - Setup: Employee with 3 days balance
   - When: Create request for 5 days
   - Then:
     - HTTP 400 Bad Request
     - Request not created
     - Balance unchanged
   - Verify: No audit log, no DB entry
3. **test_approve_with_stale_cache_performs_sync**
   - Setup: Request created 1 hour ago, cache TTL 30 min
   - When: Approve request
   - Then:
     - System calls HCM real-time API before approving
     - Uses fresh balance, not cached balance
   - Verify: HCM was called, sync_timestamp updated
4. **test_drift_detected_during_approval_conflict**
   - Setup:
     - Request created with 7 days available
     - HCM updates balance to 2 days (independent change)
   - When: Approve request (before drift sync runs)
   - Then:
     - Approval fails with BalanceConflictException
     - Request remains PENDING
     - Drift recorded in balance_history
   - Verify: Request not approved, drift flagged
5. **test_concurrent_approvals_only_one_succeeds**

- Setup: Two managers simultaneously approve same balance
- When: Both send approval requests concurrently
- Then:
  - First approval succeeds
  - Second gets version mismatch error (409 Conflict)
  - Balance only deducted once
- Verify: Only 1 approval processed, 1 failed

6. **test_cancel_approved_request_returns_balance**
   - Setup: Request approved, balance at 7 (was 10)
   - When: Cancel request
   - Then:
     - Request status = CANCELLED
     - HCM notified of cancellation
     - Balance doesn't change (cancellation doesn't restore in HCM)
   - Verify: Cancellation logged, HCM sync attempted
7. **test_reject_request_no_balance_impact**
   - Setup: Request pending, balance at 10
   - When: Reject request
   - Then:
     - Request status = REJECTED
     - Balance unchanged (never deducted)
     - Rejection reason stored
   - Verify: No HCM call made, balance unchanged
8. **test_request_with_invalid_date_range_rejected**
   - Setup: Various invalid date combinations
   - When: Create request with:
     - Start > End
     - Past dates
     - Zero-day duration
   - Then: All rejected with validation error
   - Verify: No requests created

3.2 Balance Synchronization Integration Tests (10 tests)

1. **test_batch_sync_all_employees_success**
    - Setup: Mock HCM with 3 employees' balances
    - When: POST /admin/sync/balances
    - Then:
        - All 3 employees synced
        - HTTP 202 Accepted returned
        - Sync ID provided
    - Verify: GET /admin/sync/{syncId} shows COMPLETED status
2. **test_batch_sync_partial_failure_continues**
    - Setup: Mock HCM with 3 employees, 1 fails (not found)
    - When: POST /admin/sync/balances
    - Then:
        - Sync completes despite 1 failure
        - 2 succeed, 1 in error list
    - Verify: Sync status shows 2 success, 1 failed
3. **test_batch_sync_updates_cache_and_timestamps**
    - Setup: Employee balance in HCM, not in ReadyOn cache
    - When: Batch sync called
    - Then:
        - Balance created in cache
        - readyon_cached_at timestamp set to now
        - is_stale = false
    - Verify: Database shows current timestamp
4. **test_batch_sync_detects_hcm_increase**
    - Setup:
        - Local balance: 10 days (cached 1 hour ago)
        - HCM balance: 15 days (work anniversary bonus)
    - When: Batch sync called
    - Then:
        - Local updated to 15 days
        - balance_history records increase
        - is_stale = false (HCM is authoritative)
    - Verify: History shows HCM_SYNC source

5. **test_batch_sync_detects_hcm_decrease_alerts**
    - Setup:
        - Local balance: 15 days
        - HCM balance: 10 days (unexpected decrease)
    - When: Batch sync called
    - Then:
        - Local updated to 10 days
        - balance_history records decrease with is_drift=true
        - Alert generated for HR review
    - Verify: Alert table has entry, alert type = CRITICAL
6. **test_real_time_sync_before_approval**
    - Setup: Request pending, cache outdated (2 hours old)
    - When: Approve request
    - Then:
        - Before approval, real-time sync called
        - Fresh balance fetched from HCM
        - Approval uses fresh balance
    - Verify: Sync timestamp updated to approval time
7. **test_real_time_sync_timeout_retries**
    - Setup: Mock HCM configured to timeout first call
    - When: Real-time sync called during approval
    - Then:
        - First attempt fails (timeout)
        - Retried after 500ms
        - Second attempt succeeds
    - Verify: Retry logged in audit
8. **test_real_time_sync_circuit_breaker_activates**
    - Setup: Mock HCM configured to always fail
    - When: Real-time sync called 5 times
    - Then:
        - Attempts 1-3: Retried with backoff
        - Attempt 4: Circuit breaker opens
        - Attempt 5: Rejected immediately
    - Verify: CircuitBreakerOpenException thrown

9. **test_drift_detection_finds_all_changed_balances**
   - Setup:
     - 5 employees in system
     - 2 have drift (HCM differs from local)
   - When: POST /admin/detect-drift
   - Then:
     - Returns 2 drifted balances
     - Returns 3 unchanged
   - Verify: Response accuracy
10. **test_on_demand_balance_refresh_via_api**
   - Setup: Employee balance cached, not recent
   - When: Employee calls GET /balances/{employeeId}/{location}
   - Then:
     - System checks cache staleness
     - Cache is stale, triggers real-time sync
     - Fresh balance returned to employee
   - Verify: Response includes fresh balance and sync time

3.3 Error Handling Integration Tests (8 tests)

1. **test_hcm_down_uses_cached_balance_with_warning**
   - Setup: HCM endpoint returns 500 error, cache < 24 hours old
   - When: Real-time sync called
   - Then:
     - Fallback to cached balance
     - Response includes warning flag: "balance_from_cache"
     - Request proceeds (not ideal but better than rejection)
   - Verify: Audit logs fallback reason
2. **test_hcm_down_beyond_24_hours_rejects_request**
   - Setup: HCM down, cache > 24 hours old
   - When: Approve request
   - Then:
     - Approval rejected
     - HTTP 503 Service Unavailable

        - Error message explains HCM unavailability
    - Verify: Request remains PENDING
3. **test_invalid_dimension_rejected_before_hcm_call**
    - Setup: Request for invalid employee-location combo
    - When: Create request
    - Then:
        - Rejected immediately
        - No HCM call made
    - Verify: Audit shows DIMENSION_VALIDATION_FAILED
4. **test_hcm_returns_insufficient_balance_rejection**
    - Setup: Request created (balance OK locally), HCM has different balance
    - When: Approve request
    - Then:
        - HCM returns 400 Insufficient Balance
        - Approval fails with conflict error
        - Request remains PENDING
    - Verify: Conflict logged, request not approved
5. **test_database_transaction_rollback_on_hcm_failure**
    - Setup: Request approval, balance will be decremented
    - When: HCM call fails mid-transaction
    - Then:
        - Local balance not decremented
        - Request remains PENDING
        - Request can be retried
    - Verify: Balance unchanged in DB
6. **test_data_integrity_negative_balance_prevented**
    - Setup: Corrupted state somehow exists with negative balance
    - When: System validates balance
    - Then:
        - DataIntegrityException thrown
        - Request rejected
        - Alert generated for admin
    - Verify: System self-heals

7. **test_duplicate_request_handling**
   - Setup: Employee submits same request twice within 1 second
   - When: Both requests processed
   - Then:
     - One succeeds with request ID
     - One rejected with duplicate error
   - Verify: Only 1 request in DB
8. **test_api_validation_rejects_invalid_payload**
   - Setup: Request with missing required field (employeeId)
   - When: POST /requests with incomplete payload
   - Then:
     - HTTP 400 Bad Request
     - Error details list missing field
   - Verify: No DB mutation

3.4 API Endpoint Integration Tests (12 tests)

Requests Endpoints

1. **test_post_requests_creates_request_returns_201**
   - Endpoint: POST /api/v1/requests
   - Given: Valid payload
   - When: Request submitted
   - Then: HTTP 201 Created, location header with request ID
2. **test_get_requests_id_returns_request_details**
   - Endpoint: GET /api/v1/requests/{requestId}
   - Given: Valid request exists
   - When: Called
   - Then: HTTP 200, full request details returned
3. **test_patch_requests_id_approve_approves_request**
   - Endpoint: PATCH /api/v1/requests/{requestId}/approve
   - Given: Request in PENDING state
   - When: Called
   - Then: HTTP 200, status changed to APPROVED
4. **test_patch_requests_id_approve_returns_409_on_conflict**

- Endpoint: PATCH /api/v1/requests/{requestId}/approve
- Given: Balance insufficient due to drift
- When: Called
- Then: HTTP 409 Conflict, drift details in response

5. **test_patch_requests_id_reject_rejects_request**
   - Endpoint: PATCH /api/v1/requests/{requestId}/reject
   - Given: Request in PENDING state, rejection reason
   - When: Called
   - Then: HTTP 200, status changed to REJECTED
6. **test_patch_requests_id_cancel_cancels_request**
   - Endpoint: PATCH /api/v1/requests/{requestId}/cancel
   - Given: Request in APPROVED state
   - When: Called
   - Then: HTTP 200, status changed to CANCELLED

Balance Endpoints

7. **test_get_balances_employee_location_returns_balance**
   - Endpoint: GET /api/v1/balances/{employeeId}/{locationId}?leaveType=VACATION
   - Given: Valid employee, balance exists
   - When: Called
   - Then: HTTP 200, balance details with metadata
8. **test_get_balances_employee_location_returns_404_not_found**
   - Endpoint: GET /api/v1/balances/{employeeId}/{locationId}?leaveType=VACATION
   - Given: Employee or balance doesn't exist
   - When: Called
   - Then: HTTP 404 Not Found
9. **test_get_balances_employee_returns_all_balances**
   - Endpoint: GET /api/v1/balances/{employeeId}
   - Given: Employee with 3 locations
   - When: Called
   - Then: HTTP 200, array of all balances

Admin Endpoints

10. **test_post_admin_sync_balances_starts_sync**
    - Endpoint: POST /api/v1/admin/sync/balances

- Given: Valid request
- When: Called
- Then: HTTP 202 Accepted, sync ID returned

11. **test_get_admin_sync_syncid_returns_sync_status**
    - Endpoint: GET /api/v1/admin/sync/{syncId}
    - Given: Sync in progress
    - When: Called
    - Then: HTTP 200, status and progress details
12. **test_post_admin_detect_drift_finds_drifted_balances**
    - Endpoint: POST /api/v1/admin/detect-drift
    - Given: Some balances have drifted
    - When: Called
    - Then: HTTP 200, array of drifted balances returned

---

4. End-to-End Tests (Full Scenarios)

4.1 Critical Path Scenarios (8 tests)

1. **test_e2e_happy_path_request_to_approval**
    - User Story: Employee requests 3 days vacation, manager approves
    - Steps:
        1. Create request via API
        2. Verify balance snapshot captured
        3. Approve request via API
        4. Verify balance in HCM updated
        5. Verify audit trail complete
    - Assertions:
        - Request status = APPROVED
        - HCM sync status = SYNCED
        - Balance reduced correctly
        - 3 audit entries (create, approve, balance sync)
2. **test_e2e_balance_insufficient_request_rejected**
    - User Story: Employee requests more days than available
    - Steps:
        1. Get current balance (3 days)

    2. Attempt create request for 5 days
    3. Verify rejection
  - Assertions:
    - HTTP 400 with InsufficientBalance error
    - No request created
    - Balance unchanged

3. **test_e2e_concurrent_requests_race_condition**
  - User Story: Two managers approve different requests for same balance simultaneously
  - Steps:
    1. Create 2 requests (3 days each) with 5 day balance
    2. Approve request 1
    3. Approve request 2 (concurrently)
    4. Verify one succeeds, one fails
  - Assertions:
    - Request 1 = APPROVED
    - Request 2 = PENDING (approval failed)
    - Balance reduced by 3 days only
    - Conflict logged

4. **test_e2e_hcm_balance_changes_detected**
  - User Story: Work anniversary bonus applied in HCM between request and approval
  - Steps:
    1. Create request (balance = 10, request = 3)
    2. HCM bonus: balance → 15
    3. Approve request
    4. Verify sync detected new balance
  - Assertions:
    - Request approved successfully
    - Balance updated from 10 to 15
    - Drift history recorded
    - Approval used correct balance

5. **test_e2e_batch_sync_reconciles_multiple_employees**
  - User Story: Run overnight batch sync, detect all changes
  - Steps:
    1. Seed 5 employees with balances

    2. Modify 2 in HCM mock (increase, decrease)
    3. Trigger batch sync
    4. Verify reconciliation
  - Assertions:
    - All 5 synced
    - 2 show drift detected
    - History updated
    - Alerts generated for decreases

6. **test_e2e_hcm_down_fallback_to_cache**
  - User Story: HCM maintenance window, requests still process
  - Steps:
    1. Cache recent balance
    2. Stop HCM mock
    3. Attempt to create and approve request
    4. Verify fallback behavior
  - Assertions:
    - Request created with warning
    - Approval completes with cached balance
    - Warning flag in response
    - Logged as fallback case
7. **test_e2e_cancel_request_after_approval**
  - User Story: Employee cancels approved request
  - Steps:
    1. Create and approve request
    2. Cancel request
    3. Verify HCM notified
  - Assertions:
    - Status = CANCELLED
    - Balance not restored (HCM is source of truth)
    - Cancellation logged
8. **test_e2e_full_request_lifecycle_all_states**
  - User Story: Request transitions through all possible states
  - Steps:
    1. Create request (PENDING)

2. Verify (PENDING → APPROVED transition)
3. Cancel (APPROVED → CANCELLED transition)
4. Create another (PENDING)
5. Reject (PENDING → REJECTED transition)

- Assertions:
  - All state transitions succeed
  - Invalid transitions prevented
  - Audit trail complete

4.2 Stress & Load Tests (4 tests)

1. **test_stress_1000_concurrent_balance_reads**
   - Scenario: 1000 concurrent employees reading balances
   - Target: < 100ms p95 response time
   - Assertions:
     - All requests succeed
     - Cache hit rate > 90%
     - No race conditions
2. **test_stress_100_concurrent_request_approvals**
   - Scenario: 100 managers approving requests concurrently
   - Target: No data corruption, correct conflict detection
   - Assertions:
     - Balance correctly deducted for each success
     - Conflicts detected and logged
     - Database integrity maintained
3. **test_stress_batch_sync_with_5000_employees**
   - Scenario: Batch sync for 5000 employees with network delays
   - Target: Complete in < 5 minutes
   - Assertions:
     - All employees synced
     - Errors handled gracefully
     - Database not corrupted
4. **test_stress_hcm_latency_simulation**
   - Scenario: HCM responses vary from 100ms to 5s
   - Target: All requests eventually succeed or fail gracefully

- Assertions:
  - Retries kick in appropriately
  - Circuit breaker prevents cascade failures
  - No hanging requests

---

5. Mock HCM Specifications

5.1 Mock HCM Endpoints

GET /hcm/balances/{employeeId}/{locationId}

```
Request:
- employeeId: String
- locationId: String
- leaveType: String (query param)

Response (200 OK):
{
  "employeeId": "EMP-001",
  "locationId": "LOC-NYC",
  "leaveType": "VACATION",
  "availableBalance": 10,
  "usedBalance": 5,
  "carryoverBalance": 2,
  "balanceYear": 2026,
  "lastUpdatedAt": "2026-04-24T10:00:00Z"
}

Error Responses:
- 400 Bad Request: Invalid dimensions
- 404 Not Found: Employee/location not found
- 500 Internal Server Error: Simulated HCM error
```

POST /hcm/time-off-requests

```
Request:
{
  "employeeId": "EMP-001",
  "locationId": "LOC-NYC",
  "leaveType": "VACATION",
  "startDate": "2026-05-01",
  "endDate": "2026-05-03",
  "days": 3
}

Response (200 OK):
{
  "hcmRequestId": "HCM-98765",
  "status": "APPROVED",
  "approvedAt": "2026-04-24T10:00:00Z"
}

Error Responses:
- 400 Bad Request: Invalid request
- 403 Forbidden: Insufficient balance
- 404 Not Found: Invalid dimension
```

POST /hcm/sync/balances

```
Request:
{
  "employeeIds": ["EMP-001", "EMP-002"]
}

Response (200 OK):
{
  "balances": [
    {
      "employeeId": "EMP-001",
      "locationId": "LOC-NYC",
      "leaveType": "VACATION",
      "availableBalance": 10,
      "lastUpdatedAt": "2026-04-24T10:00:00Z"
    }
  ]
}
```

5.2 Mock HCM Behaviors

1. **Normal Operation**: Returns balances as configured
2. **Timeout Simulation**: Delay response by 6+ seconds (triggers timeout)
3. **Partial Failure**: Return error for specific employee, success for others
4. **Drift Simulation**: Update balance between calls to simulate work anniversary
5. **Circuit Breaker Test**: Return error for first N calls, succeed on N+1
6. **Invalid Dimension**: Return 404 for non-existent employee-location combo
7. **Insufficient Balance**: Return 403 when request exceeds balance

---

6. Test Data & Fixtures

6.1 Employee Fixtures

```
EMP-001: NYC location, VACATION + SICK, 10 days + 5 days
EMP-002: NYC location, VACATION only, 3 days
EMP-003: BOSTON location, VACATION + SICK + PTO, full suite
EMP-004: Invalid employee (for negative tests)
```

6.2 Balance Fixtures

```
Pre-load 10 employees with varying balances
- 2 with sufficient balance (10+ days)
- 3 with low balance (2-5 days)
- 2 with zero balance
- 2 with mixed (sufficient for one type, insufficient for another)
- 1 with corrupted data (negative balance, for data integrity test)
```

6.3 Request Fixtures

```
- Pending requests (not yet approved)
- Approved requests (for cancel/update tests)
- Rejected requests (for state transition tests)
- Past requests (for history/audit tests)
```

---

7. Test Execution Strategy

7.1 Local Development

```
# Run all tests
npm test

# Run unit tests only
npm test -- --testPathPattern="\.unit\.ts$"

# Run integration tests only
npm test -- --testPathPattern="\.integration\.ts$"

# Run specific test suite
npm test -- TimeOffRequestService

# Run with coverage
npm test -- --coverage
```

7.2 CI/CD Pipeline

```
1. Unit tests (fast fail, 2 minutes)
2. Integration tests (5 minutes)
3. E2E tests (10 minutes)
4. Load tests (5 minutes, optional)
5. Coverage report (fail if < 80%)
```

7.3 Coverage Goals

- Overall: > 80%
- Critical paths: 100% (request approval, balance validation, sync)
- Error handling: > 90%
- Data access layer: > 85%

---

8. Test Documentation

Each test will document:

1. Test ID (TC-001, etc.)
2. Title
3. Given/When/Then format
4. Setup steps
5. Assertions
6. Expected side effects (DB, audit logs, HCM calls)

7. Teardown requirements
8. Dependencies on other tests

---

9. Success Criteria

1. ✅ All 100+ test cases pass
2. ✅ Code coverage > 80%
3. ✅ Critical path tests (request approval, balance validation) at 100%
4. ✅ All race condition scenarios tested
5. ✅ All error scenarios tested
6. ✅ Mock HCM behaviors comprehensive
7. ✅ Load tests pass (stress scenarios)
8. ✅ Audit trail verified for all state changes